

HDOC DAY 20 PROGRAMMING IN C

Problem

Prepare an algorithm for each operation and a test-case table covering valid, invalid, and boundary inputs. Then

develop, compile, and execute a menu-driven modular C project that calculates the geometrical measurements

and physical properties of selected solids. Test the program using the prepared test cases.

[Note: Use only fundamental data types, functions with call by value, selection statements, and loops. Do not use

recursion, arrays, pointer variables, pointer parameters, structures, unions, enumerations, typedef, or functions

from math.h. Assume that all input values and calculated results are within the ranges supported by their respective

data types.]

Operations

- Find the total surface area and volume of a sphere [double sphere_area(double radius), double

sphere_volume(double radius)]

- Find the total surface area and volume of a cylinder [double cylinder_area(double radius, double height),

double cylinder_volume(double radius, double height)]

- Find the total surface area and volume of a cone [double cone_area(double radius, double height), double

cone_volume(double radius, double height)]

- Find the total surface area and volume of a cuboid [double cuboid_area(double length, double breadth,

double height), double cuboid_volume(double length, double breadth, double height)]

- Find the base area of cylinder and cone [double circular_base_area(double radius)]

HDOC DAY 20 REPORT

- Find the base area of cuboid [double cuboid_base_area(double length, double breadth)]
- Calculate the mass of a solid from its density and volume [double calculate_mass(double density)]
- Calculate the weight of a solid [double calculate_weight()]
- Calculate the gravitational potential energy of a solid at a given elevation [double potential_energy(double elevation)]
- Calculate the pressure exerted by a cylinder, cone, or cuboid resting on its base [double calculate_pressure(double density)]
- Calculate the buoyant force on a completely submerged solid [double buoyant_force(double fluid_density)]
- Determine whether a completely immersed solid sinks, floats, or remains suspended [int determine_buoyancy(double solid_density, double fluid_density)]

Organization of the project

Module Functions/Responsibility

main.c Displays the main menu, obtains common inputs, calls the required functions, and displays results

geometry.c Calculates surface area, volume, base area, and slant height

physics.c Obtains the solid type and required dimensions for physics operations and performs the corresponding calculations

math_utils.c Calculates square root iteratively using the Newton-Raphson method

solid.h Contains the header guard, symbolic constants, and external function declarations

Header File Outline

- Use #ifndef, #define, and #endif to create a header guard.
- Define PI as 3.1416, GRAVITY as 9.8, and TOLERANCE as 0.000001.

HDOC DAY 20 REPORT

- Define the solid choices SPHERE, CYLINDER, CONE, and CUBOID with the values 1, 2, 3, and 4,

respectively.

- Define the buoyancy results FLOATS, SUSPENDED, and SINKS with the values -1, 0, and 1, respectively.

- Include the external declarations of all functions defined in geometry.c, physics.c, and math_utils.c.

Outline of main()

- Use a do-while loop to repeat the menu until the Exit option is selected.
- Display the menu: (1) Surface area, (2) Volume, (3) Base Area, (4) Mass, (5) Weight, (6) Gravitational

potential energy, (7) Pressure on the base, (8) Buoyant force, (9) Buoyancy behavior, and (10) Exit.

o For operations involving geometry, display a submenu to select the solid.

o Supported Solids: Sphere, Cylinder, Cone, Cuboid

- Validate each input immediately after it is obtained, whether it is obtained in main() or another function.

Solid choices must be within the specified range. Dimensions and densities must be greater than zero, and

elevation must not be negative. If an invalid value is entered, display an appropriate error message and

request the value again.

- For the Base Area option, permit only cylinder, cone, and cuboid. If the user selects sphere, display an

appropriate message and return to the main menu.

- Use SI units throughout the project. Enter dimensions and elevation in metres and densities in kilograms

per cubic metre. Display area in square metres, volume in cubic metres, mass in kilograms, weight and

buoyant force in newtons, pressure in pascals, and potential energy in joules.

- Use switch-case to call the corresponding function and display the result.

HDOC DAY 20 REPORT

- Do not repeat geometrical or physical formulas in main().

Outline of geometry.c

- sphere_area()
 - o Calculate and return $4 * \text{PI} * \text{radius} * \text{radius}$.
- sphere_volume()
 - o Calculate and return $(4.0 / 3.0) * \text{PI} * \text{radius} * \text{radius} * \text{radius}$.
- cylinder_area()
 - o Calculate and return $2 * \text{PI} * \text{radius} * (\text{radius} + \text{height})$.
- cylinder_volume()
 - o Calculate and return $\text{PI} * \text{radius} * \text{radius} * \text{height}$.
- cone_area()
 - o Calculate the slant_height as the square root of $\text{radius} * \text{radius} + \text{height} * \text{height}$ by calling
calculate_square_root().
 - o Calculate and return $\text{PI} * \text{radius} * (\text{radius} + \text{slant_height})$.
- cone_volume()
 - o Calculate and return $(1.0 / 3.0) * \text{PI} * \text{radius} * \text{radius} * \text{height}$.
- cuboid_area()
 - o Calculate and return $2 * (\text{length} * \text{breadth} + \text{breadth} * \text{height} + \text{length} * \text{height})$.
- cuboid_volume()
 - o Calculate and return $\text{length} * \text{breadth} * \text{height}$.
- circular_base_area()
 - o Calculate and return $\text{PI} * \text{radius} * \text{radius}$.
 - o Use it for cylinders and cones.
- cuboid_base_area()
 - o Calculate and return $\text{length} * \text{breadth}$.
- Note: As the ideal geometrical base area for a sphere is not defined by the given dimensions, it cannot be
calculated.

HDOC DAY 20 REPORT

Outline of physics.c

- calculate_mass()
 - o Get the type of solid (sphere/cone/cylinder/cuboid).
 - o Use appropriate function to calculate the volume of the solid. i.e. sphere_volume() or cone_volume() or cylinder_volume() or cuboid_volume().
 - o Use $\text{mass} = \text{density} * \text{volume}$ and return the mass in kilograms.
- calculate_weight()
 - o Obtain mass from calculate_mass().
 - o Use $\text{weight} = \text{mass} * \text{GRAVITY}$ and return the weight in newtons.
- potential_energy()
 - o Obtain mass from calculate_mass().
 - o Use $\text{potential energy} = \text{mass} * \text{GRAVITY} * \text{elevation}$ and return the energy in joules.
- calculate_pressure()
 - o Get the type of solid (cone/cylinder/cuboid).
 - Obtain contact_area from circular_base_area() for cylinder or cone, and from cuboid_base_area() for cuboid.
 - Obtain volume from appropriate function cone_volume() or cylinder_volume() or cuboid_volume().
 - o Use $\text{pressure} = (\text{density} * \text{volume} * \text{GRAVITY}) / \text{contact_area}$ and return the pressure in pascals.
 - o Note: Do not calculate pressure for a sphere because the ideal geometrical contact area is not defined by the given dimensions.
- buoyant_force()
 - o Get the type of solid (sphere/cone/cylinder/cuboid).
 - Obtain volume from appropriate function sphere_volume() or cone_volume() or cylinder_volume() or cuboid_volume().
 - o Use $\text{buoyant force} = \text{fluid_density} * \text{volume} * \text{GRAVITY}$ and return the force in newtons.
- determine_buoyancy()

HDOC DAY 20 REPORT

- o Return SINKS if solid_density is greater than fluid_density.
- o Return FLOATS if solid_density is less than fluid_density.
- o Return SUSPENDED if the two densities are equal.
- o Display the corresponding message in main().

Outline of math_utils.c

- calculate_square_root()
 - o Do not use sqrt(), fabs(), or any other function from math.h.
 - o If number is negative, return -1 to indicate that its real square root cannot be calculated.
 - o If number is zero, return 0.
 - o For a positive number, initialize old_value with number.
 - o Use a do-while loop to perform the following steps:
 - Calculate a better approximation using
 - $\text{new_value} = 0.5 * (\text{old_value} + \text{number} / \text{old_value})$
 - Calculate the difference between the new and previous approximations:
 - $\text{difference} = \text{new_value} - \text{old_value}$
 - If difference is negative, change it to its positive value.
 - Assign new_value to old_value so that the new approximation is used in the next iteration.
 - o Continue the loop while difference is greater than or equal to TOLERANCE.
 - o Return new_value as the approximate square root of number

HDOC DAY 20 REPORT

C PROGRAM CODE

```
#ifndef SOLID_H
```

```
#define SOLID_H
```

```
#define PI 3.1416
```

```
#define GRAVITY 9.8
```

```
#define TOLERANCE 0.000001
```

```
#define SPHERE 1
```

```
#define CYLINDER 2
```

```
#define CONE 3
```

```
#define CUBOID 4
```

```
#define FLOATS -1
```

```
#define SUSPENDED 0
```

```
#define SINKS 1
```

```
double sphere_area(double radius);
```

```
double sphere_volume(double radius);
```

```
double cylinder_area(double radius, double height);
```

```
double cylinder_volume(double radius, double height);
```

```
double cone_area(double radius, double height);
```

```
double cone_volume(double radius, double height);
```

```
double cuboid_area(double length, double breadth, double height);
```

```
double cuboid_volume(double length, double breadth, double height);
```

HDOC DAY 20 REPORT

```
double circular_base_area(double radius);

double cuboid_base_area(double length, double breadth);


double calculate_mass(double density);

double calculate_weight();

double potential_energy(double elevation);


double calculate_pressure(double density);

double buoyant_force(double fluid_density);


int determine_buoyancy(double solid_density, double fluid_density);


double calculate_square_root(double number);


#endif

Math_utils.c

#include "solid.h"


double calculate_square_root(double number)
{
    double old_value;

    double new_value;

    double difference;


    if (number < 0)

        return -1;
```

HDOC DAY 20 REPORT

```
if (number == 0)
    return 0;

old_value = number;

do
{
    new_value = 0.5 * (old_value + number / old_value);

    difference = new_value - old_value;

    if (difference < 0)
        difference = -difference;

    old_value = new_value;

} while (difference >= TOLERANCE);

return new_value;
}
```

Geomectry.c

```
#include "solid.h"
```

```
double sphere_area(double radius)
{
    return 4 * PI * radius * radius;
}
```

HDOC DAY 20 REPORT

```
double sphere_volume(double radius)
{
    return (4.0 / 3.0) * PI * radius * radius * radius;
}
```

```
double cylinder_area(double radius, double height)
{
    return 2 * PI * radius * (radius + height);
}
```

```
double cylinder_volume(double radius, double height)
{
    return PI * radius * radius * height;
}
```

```
double cone_area(double radius, double height)
{
    double slant_height;

    slant_height = calculate_square_root(
        radius * radius + height * height
    );

    return PI * radius * (radius + slant_height);
}
```

```
double cone_volume(double radius, double height)
{

```

HDOC DAY 20 REPORT

```
    return (1.0 / 3.0) * PI * radius * radius * height;
}
```

```
double cuboid_area(double length, double breadth, double height)
{
    return 2 * (length * breadth +
                breadth * height +
                length * height);
}
```

```
double cuboid_volume(double length, double breadth, double height)
{
    return length * breadth * height;
}
```

```
double circular_base_area(double radius)
{
    return PI * radius * radius;
}
```

```
double cuboid_base_area(double length, double breadth)
{
    return length * breadth;
}
```

[physics.c](#)

```
#include <stdio.h>
```

```
#include "solid.h"
```

HDOC DAY 20 REPORT

```
double calculate_mass(double density)
{
    int choice;

    double radius, height;

    double length, breadth;

    double volume;

    do
    {
        printf("\nSelect solid:\n");
        printf("1. Sphere\n");
        printf("2. Cylinder\n");
        printf("3. Cone\n");
        printf("4. Cuboid\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        if (choice < SPHERE || choice > CUBOID)
            printf("Invalid choice. Try again.\n");

    } while (choice < SPHERE || choice > CUBOID);

    switch (choice)
    {
        case SPHERE:
            do
            {
                printf("Enter radius (m): ");
```

HDOC DAY 20 REPORT

```
scanf("%lf", &radius);
```

```
if (radius <= 0)
```

```
    printf("Radius must be greater than zero.\n");
```

```
} while (radius <= 0);
```

```
volume = sphere_volume(radius);
```

```
break;
```

```
case CYLINDER:
```

```
do
```

```
{
```

```
    printf("Enter radius (m): ");
```

```
    scanf("%lf", &radius);
```

```
    if (radius <= 0)
```

```
        printf("Radius must be greater than zero.\n");
```

```
} while (radius <= 0);
```

```
do
```

```
{
```

```
    printf("Enter height (m): ");
```

```
    scanf("%lf", &height);
```

```
    if (height <= 0)
```

```
        printf("Height must be greater than zero.\n");
```

HDOC DAY 20 REPORT

```
} while (height <= 0);
```

```
volume = cylinder_volume(radius, height);
```

```
break;
```

```
case CONE:
```

```
do
```

```
{
```

```
    printf("Enter radius (m): ");
```

```
    scanf("%lf", &radius);
```

```
    if (radius <= 0)
```

```
        printf("Radius must be greater than zero.\n");
```

```
} while (radius <= 0);
```

```
do
```

```
{
```

```
    printf("Enter height (m): ");
```

```
    scanf("%lf", &height);
```

```
    if (height <= 0)
```

```
        printf("Height must be greater than zero.\n");
```

```
} while (height <= 0);
```

```
volume = cone_volume(radius, height);
```

HDOC DAY 20 REPORT

```
break;
```

```
case CUBOID:
```

```
do
```

```
{
```

```
    printf("Enter length (m): ");
```

```
    scanf("%lf", &length);
```

```
    if (length <= 0)
```

```
        printf("Length must be greater than zero.\n");
```

```
} while (length <= 0);
```

```
do
```

```
{
```

```
    printf("Enter breadth (m): ");
```

```
    scanf("%lf", &breadth);
```

```
    if (breadth <= 0)
```

```
        printf("Breadth must be greater than zero.\n");
```

```
} while (breadth <= 0);
```

```
do
```

```
{
```

```
    printf("Enter height (m): ");
```

```
    scanf("%lf", &height);
```

HDOC DAY 20 REPORT

```
        if (height <= 0)
            printf("Height must be greater than zero.\n");

    } while (height <= 0);

    volume = cuboid_volume(length, breadth, height);
    break;
}

return density * volume;
}

double calculate_weight()
{
    double density;
    double mass;

    do
    {
        printf("Enter density (kg/m^3): ");
        scanf("%lf", &density);

        if (density <= 0)
            printf("Density must be greater than zero.\n");

    } while (density <= 0);

    mass = calculate_mass(density);
```

HDOC DAY 20 REPORT

```
    return mass * GRAVITY;  
}
```

```
double potential_energy(double elevation)
```

```
{  
    double density;  
    double mass;  
  
    do  
    {  
        printf("Enter density (kg/m^3): ");  
        scanf("%lf", &density);  
  
        if (density <= 0)  
            printf("Density must be greater than zero.\n");  
    } while (density <= 0);  
  
    mass = calculate_mass(density);  
  
    return mass * GRAVITY * elevation;  
}
```

```
double calculate_pressure(double density)
```

```
{  
    int choice;  
    double radius, height;
```

HDOC DAY 20 REPORT

double length, breadth;

double contact_area;

double volume;

do

{

printf("\nSelect solid:\n");

printf("1. Cylinder\n");

printf("2. Cone\n");

printf("3. Cuboid\n");

printf("Enter choice: ");

scanf("%d", &choice);

if (choice < 1 || choice > 3)

printf("Invalid choice. Try again.\n");

} while (choice < 1 || choice > 3);

if (choice == 1)

{

do

{

printf("Enter radius (m): ");

scanf("%lf", &radius);

} while (radius <= 0);

do

{

HDOC DAY 20 REPORT

```
        printf("Enter height (m): ");
        scanf("%lf", &height);
    } while (height <= 0);

    contact_area = circular_base_area(radius);
    volume = cylinder_volume(radius, height);
}
else if (choice == 2)
{
    do
    {
        printf("Enter radius (m): ");
        scanf("%lf", &radius);
    } while (radius <= 0);

    do
    {
        printf("Enter height (m): ");
        scanf("%lf", &height);
    } while (height <= 0);

    contact_area = circular_base_area(radius);
    volume = cone_volume(radius, height);
}
else
{
    do
    {
```

HDOC DAY 20 REPORT

```
    printf("Enter length (m): ");
    scanf("%lf", &length);
} while (length <= 0);

do
{
    printf("Enter breadth (m): ");
    scanf("%lf", &breadth);
} while (breadth <= 0);

do
{
    printf("Enter height (m): ");
    scanf("%lf", &height);
} while (height <= 0);

contact_area = cuboid_base_area(length, breadth);
volume = cuboid_volume(length, breadth, height);
}

return (density * volume * GRAVITY) / contact_area;
}

double buoyant_force(double fluid_density)
{
    int choice;
    double radius, height;
    double length, breadth;
```

HDOC DAY 20 REPORT

```
double volume;

do
{
    printf("\nSelect solid:\n");
    printf("1. Sphere\n");
    printf("2. Cylinder\n");
    printf("3. Cone\n");
    printf("4. Cuboid\n");
    printf("Enter choice: ");
    scanf("%d", &choice);

} while (choice < SPHERE || choice > CUBOID);

if (choice == SPHERE)
{
    do
    {
        printf("Enter radius (m): ");
        scanf("%lf", &radius);
    } while (radius <= 0);

    volume = sphere_volume(radius);
}
else if (choice == CYLINDER)
{
    do
    {
```

HDOC DAY 20 REPORT

```
        printf("Enter radius (m): ");
        scanf("%lf", &radius);
    } while (radius <= 0);

    do
    {
        printf("Enter height (m): ");
        scanf("%lf", &height);
    } while (height <= 0);

    volume = cylinder_volume(radius, height);
}

else if (choice == CONE)
{
    do
    {
        printf("Enter radius (m): ");
        scanf("%lf", &radius);
    } while (radius <= 0);

    do
    {
        printf("Enter height (m): ");
        scanf("%lf", &height);
    } while (height <= 0);

    volume = cone_volume(radius, height);
}
```

HDOC DAY 20 REPORT

```
else
{
    do
    {
        printf("Enter length (m): ");
        scanf("%lf", &length);
    } while (length <= 0);

    do
    {
        printf("Enter breadth (m): ");
        scanf("%lf", &breadth);
    } while (breadth <= 0);

    do
    {
        printf("Enter height (m): ");
        scanf("%lf", &height);
    } while (height <= 0);

    volume = cuboid_volume(length, breadth, height);
}

return fluid_density * volume * GRAVITY;
}

int determine_buoyancy(double solid_density, double fluid_density)
{
```

HDOC DAY 20 REPORT

```
    if (solid_density > fluid_density)
        return SINKS;
    else if (solid_density < fluid_density)
        return FLOATS;
    else
        return SUSPENDED;
}
```

main.c

```
#include <stdio.h>
```

```
#include "solid.h"
```

```
int main()
```

```
{
```

```
    int choice;
```

```
    int solid;
```

```
    int result;
```

```
    double radius, height;
```

```
    double length, breadth;
```

```
    double density;
```

```
    double fluid_density;
```

```
    double elevation;
```

```
    double answer;
```

```
do
```

```
{
```

```
    printf("\n===== SOLID PHYSICS & GEOMETRY =====\n");
```

```
    printf("1. Surface Area\n");
```

HDOC DAY 20 REPORT

```
printf("2. Volume\n");
printf("3. Base Area\n");
printf("4. Mass\n");
printf("5. Weight\n");
printf("6. Gravitational Potential Energy\n");
printf("7. Pressure on the Base\n");
printf("8. Buoyant Force\n");
printf("9. Buoyancy Behavior\n");
printf("10. Exit\n");
printf("Enter choice: ");
scanf("%d", &choice);

switch (choice)
{
    case 1:
        printf("\nSelect solid:\n");
        printf("1. Sphere\n2. Cylinder\n3. Cone\n4. Cuboid\n");
        printf("Enter choice: ");
        scanf("%d", &solid);

        if (solid == SPHERE)
        {
            do
            {
                printf("Enter radius: ");
                scanf("%lf", &radius);

            } while (radius <= 0);
```

HDOC DAY 20 REPORT

```
    answer = sphere_area(radius);
}
else if (solid == CYLINDER)
{
    do
    {
        printf("Enter radius: ");
        scanf("%lf", &radius);
    } while (radius <= 0);

    do
    {
        printf("Enter height: ");
        scanf("%lf", &height);
    } while (height <= 0);

    answer = cylinder_area(radius, height);
}
else if (solid == CONE)
{
    do
    {
        printf("Enter radius: ");
        scanf("%lf", &radius);
    } while (radius <= 0);

    do
    {
```

HDOC DAY 20 REPORT

```
        printf("Enter height: ");
        scanf("%lf", &height);
    } while (height <= 0);

    answer = cone_area(radius, height);
}
else if (solid == CUBOID)
{
    do
    {
        printf("Enter length: ");
        scanf("%lf", &length);
    } while (length <= 0);

    do
    {
        printf("Enter breadth: ");
        scanf("%lf", &breadth);
    } while (breadth <= 0);

    do
    {
        printf("Enter height: ");
        scanf("%lf", &height);
    } while (height <= 0);

    answer = cuboid_area(length, breadth, height);
}
```

HDOC DAY 20 REPORT

```
else
{
    printf("Invalid solid.\n");
    break;
}

printf("Surface Area = %.2lf m^2\n", answer);
break;
```

case 2:

```
printf("\nSelect solid:\n");
printf("1. Sphere\n2. Cylinder\n3. Cone\n4. Cuboid\n");
printf("Enter choice: ");
scanf("%d", &solid);
```

```
if (solid == SPHERE)
{
    do
    {
        printf("Enter radius: ");
        scanf("%lf", &radius);
    } while (radius <= 0);
```

```
    answer = sphere_volume(radius);
}
```

```
else if (solid == CYLINDER)
{
    do
```

HDOC DAY 20 REPORT

```
{  
    printf("Enter radius: ");  
    scanf("%lf", &radius);  
} while (radius <= 0);  
  
do  
{  
    printf("Enter height: ");  
    scanf("%lf", &height);  
} while (height <= 0);  
  
    answer = cylinder_volume(radius, height);  
}  
else if (solid == CONE)  
{  
    do  
    {  
        printf("Enter radius: ");  
        scanf("%lf", &radius);  
    } while (radius <= 0);  
  
    do  
    {  
        printf("Enter height: ");  
        scanf("%lf", &height);  
    } while (height <= 0);  
  
    answer = cone_volume(radius, height);
```

HDOC DAY 20 REPORT

```
}  
else if (solid == CUBOID)  
{  
    do  
    {  
        printf("Enter length: ");  
        scanf("%lf", &length);  
    } while (length <= 0);  
  
    do  
    {  
        printf("Enter breadth: ");  
        scanf("%lf", &breadth);  
    } while (breadth <= 0);  
  
    do  
    {  
        printf("Enter height: ");  
        scanf("%lf", &height);  
    } while (height <= 0);  
  
    answer = cuboid_volume(length, breadth, height);  
}  
else  
{  
    printf("Invalid solid.\n");  
    break;  
}
```

HDOC DAY 20 REPORT

```
printf("Volume = %.2lf m^3\n", answer);  
break;
```

case 3:

```
printf("\n1. Cylinder\n2. Cone\n3. Cuboid\n");  
printf("Enter choice: ");  
scanf("%d", &solid);
```

```
if (solid == CYLINDER || solid == CONE)  
{  
    do  
    {  
        printf("Enter radius: ");  
        scanf("%lf", &radius);  
    } while (radius <= 0);
```

```
    answer = circular_base_area(radius);  
}
```

```
else if (solid == CUBOID)  
{  
    do  
    {  
        printf("Enter length: ");  
        scanf("%lf", &length);  
    } while (length <= 0);
```

```
    do
```

HDOC DAY 20 REPORT

```
{
    printf("Enter breadth: ");
    scanf("%lf", &breadth);
} while (breadth <= 0);

    answer = cuboid_base_area(length, breadth);
}
else
{
    printf("Sphere does not have a defined base area.\n");
    break;
}

printf("Base Area = %.2lf m^2\n", answer);
break;

case 4:
do
{
    printf("Enter density (kg/m^3): ");
    scanf("%lf", &density);
} while (density <= 0);

    answer = calculate_mass(density);
    printf("Mass = %.2lf kg\n", answer);
    break;

case 5:
```

HDOC DAY 20 REPORT

```
answer = calculate_weight();  
  
printf("Weight = %.2lf N\n", answer);  
  
break;
```

case 6:

```
do  
  
{  
  
    printf("Enter elevation (m): ");  
  
    scanf("%lf", &elevation);  
  
  
    if (elevation < 0)  
  
        printf("Elevation cannot be negative.\n");  
  
} while (elevation < 0);
```

```
answer = potential_energy(elevation);  
  
printf("Potential Energy = %.2lf J\n", answer);  
  
break;
```

case 7:

```
do  
  
{  
  
    printf("Enter density (kg/m^3): ");  
  
    scanf("%lf", &density);  
  
} while (density <= 0);
```

```
answer = calculate_pressure(density);  
  
printf("Pressure = %.2lf Pa\n", answer);
```

HDOC DAY 20 REPORT

```
break;
```

```
case 8:
```

```
do
```

```
{
```

```
    printf("Enter fluid density (kg/m^3): ");
```

```
    scanf("%lf", &fluid_density);
```

```
} while (fluid_density <= 0);
```

```
answer = buoyant_force(fluid_density);
```

```
printf("Buoyant Force = %.2lf N\n", answer);
```

```
break;
```

```
case 9:
```

```
do
```

```
{
```

```
    printf("Enter solid density (kg/m^3): ");
```

```
    scanf("%lf", &density);
```

```
} while (density <= 0);
```

```
do
```

```
{
```

```
    printf("Enter fluid density (kg/m^3): ");
```

```
    scanf("%lf", &fluid_density);
```

```
} while (fluid_density <= 0);
```

```
result = determine_buoyancy(density, fluid_density);
```

HDOC DAY 20 REPORT

```
        if (result == SINKS)

            printf("The solid sinks.\n");

        else if (result == FLOATS)

            printf("The solid floats.\n");

        else

            printf("The solid remains suspended.\n");


        break;


    case 10:

        printf("Exiting program...\n");

        break;


    default:

        printf("Invalid menu choice.\n");

    }


} while (choice != 10);


return 0;

}
```